Task for the Master's Thesis

Danje Petersen

November 6, 2025

1 Motivation

Modern computers come with a set of processors that can run simultaneously. To utilize this for their advantage, developers started to design their programs to use multiple concurrently running threads. Since these threads might still operate on the same memory it is possible to encounter problems like data-races when executing these programs. Data races occur when two or more threads access the same memory space and at least one of them uses a write operation. The classic example is the following:

```
Init:x=y=0;

x=1; || y=1;
a=y; || b=x;
```

Figure 1: Example of two threads using the same variables. Depending on how the threads are scheduled, the resulting values could feature any combination of 1 and 0 for the variables a and b.

Since this might lead to inconsistent results and therefore unpredictable behaviour, programming languages and cpu architectures provide memory consistency models. If the developer confines themselves to these rules, they get certain guarantees in return. These memory consistency models describe how threads are allowed to interact with each other when it comes to shared memory. As can be seen in the example, not having a memory consistency model can have serious consequences for the execution of programs. But having a memory consistency model will not automatically get rid of all problems. It is not guaranteed for example that a given model will not produce out-of-thin-air values (oota) [**BoeDem14**]. Therefore it is an important field of research, especially for software used for critical infrastructure and projects.

2 Current Research

In the field of memory consistency models current research is aimed at a variety of issues. One obvious issue is model checking programs for a given memory model [12, 8]. Others might focus on the models themselves and their attributes [10] or other kinds of models like operational ones [5, 3]. Additionally, researchers are looking at ways of comparing [13, 11] and verifying [7, 2, 6] memory consistency models. The verification comes down to five different tasks:

- check certain properties (like no-oota)
- check properties referring to semantics (like barrier monotonicity)
- compare memory consistency models
- create litmus tests
- verify compiler transformations

In 2014 Alglave et al. proposed an axiomatic framework for modelling weak memory and the corresponding language CAT [1]. Axiomatic models work by constraining relations on so called events. The events are the memory accesses in a program like reads and writes.

$$
\begin{aligned}
\langle cat \rangle &:= \langle constraint \rangle \mid \langle cat \rangle\ \langle cat \rangle \\
\langle constraint \rangle &:= \texttt{acyclic}\ \langle rel \rangle \mid \texttt{irreflexive}\ \langle rel \rangle \\
\langle rel \rangle &:= \langle base_rel \rangle \mid \langle rel \rangle.\langle rel \rangle \mid \langle rel \rangle^* \mid \langle rel \rangle \cup \langle rel \rangle \mid \langle rel \rangle \cap \langle rel \rangle \\
&\quad \mid \langle rel \rangle^{-1} \mid \langle rel \rangle^+ \mid \langle rel \rangle? \mid \langle base_set \rangle \times \langle base_set \rangle \mid \langle rel \rangle \setminus \langle rel \rangle \mid \sim \langle rel \rangle \\
\langle base_rel \rangle &:= \text{po} \mid \text{rf} \mid \text{rmw} \mid \text{loc} \mid \text{ext} \mid \text{id} \mid \emptyset \\
\langle base_set \rangle &:= \text{R} \mid \text{W} \mid \text{F} \mid \text{B} \mid \text{M} \mid \text{IW} \mid \text{E}
\end{aligned}
$$

Figure 2: The CAT language when including the red coloured terms, when excluding them it is the fragment called rational CAT. $\langle rel \rangle^+$ is the transitive closure, $\langle rel \rangle?$ the reflexive closure, $\langle rel \rangle^*$ the reflexive and transitive closure, $\sim \langle rel \rangle$ the complement [1].

Some very basic examples for these relations are the *program order* (po) which represents the sequence of events of a single thread and *read-from* (rf), displaying for every read which write determined the value it sees. Upon these and some more base relations, memory consistency models can be expressed with CAT as can be seen in fig. 2.

While the tool that Alglave et al. developed allows to test certain programs in a given memory model, verifying these models is a different issue. Verification of the memory consistency model means checking if a given model guarantees certain properties for every program

that can be executed under it. For this purpose, the memory model in question will be compared to a model that is defined by just that property. The first tool to tackle this issue was KATER [7] which covered only a fragment of CAT that could be transformed into Kleene algebra with tests. This fragment excluded set difference, complement, intersections, converses and the construction of relations from sets.
The next year Koutsouridis et al. presented KATI [9] which is able to handle converses and some intersections.

The first tool that addressed all intersections as well as inverses and construction of relations from sets (called rational CAT, see fig. 2) was developed by the supervisors of this master's thesis in 2025 [4]. In their yet to be published work, they presented a logical theory of relations where the unsatisfiability of a formula is equivalent to the model fulfilling a condition. By transforming the question of $M \subseteq M_{prop}$ into $\overline{M} \cap M_{prop}$ and then a formula $\neg\phi_M \wedge \phi_{M_{prop}}$ it proves if the given model is stronger than the condition that is to be tested. The formula is then decomposed until there exists a satisfying model for the formula or it is deemed unsatisfiable.

Today there are many open questions in this field of research. Is it possible to automatically verify the whole of the CAT language or at least more than the rational CAT fragment? Can the popular models written in CAT be transformed into ones that are written in rational CAT? Is it possible to verify compiler transformations?

3 Task

The decomposition used in [4] might result in infinite paths when unrolling Kleene stars. This leads to the authors using a cyclic proof system that returns to a node in the decomposition tree when it is equal to the current node. The task for this master's thesis is to replace this cyclic proof system with one that uses invariants and is neither cyclic nor infinite. Additionally the new proof system is supposed to directly handle inclusion between memory models and does not need to resolve them to the event level. The student is supposed to create a proof system that works reasonably well and ideally proof it's completeness and soundness.

References

[1] Jade Alglave, Luc Maranget, and Michael Tautschnig. "Herding Cats: Modelling, simulation, testing and data mining for weak memory". In: *ACM Transactions on Programming Languages and Systems* 36.2 (2014), pp. 1–74. ISSN: 0164-0925. DOI: 10.1145/2627752.

[2] P. S. Andrianov et al. "Software Model Checking for Memory Consistency Verification". In: *Programming and Computer Software* 51.5 (2025), pp. 305–313. ISSN: 1608-3261. DOI: 10.1134/S0361768825700185. URL: https://link.springer.com/article/10.1134/s0361768825700185.

[3] Simon Doherty et al. "Unifying Operational Weak Memory Verification: An Axiomatic Approach". In: *ACM Transactions on Computational Logic* 23.4 (2022), pp. 1–39. ISSN: 1529-3785. DOI: 10.1145/3545117.

[4] Jan Grünke and Roland Meyer. "RAT-CAT-SAT: Model Checking Memory Consistency Models". In: *unpublished* (2025).

[5] Dan Iorga, John Wickerson, and Alastair F. Donaldson. "Simulating Operational Memory Models Using Off-the-Shelf Program Analysis Tools". In: *IEEE Transactions on Software Engineering* 49.12 (2023), pp. 5084–5102. ISSN: 0098-5589. DOI: 10.1109/TSE.2023.3326056.

[6] Jake Kirkham et al. "Foundations of empirical memory consistency testing". In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA (2020), pp. 1–29. DOI: 10.1145/3428294.

[7] Michalis Kokologiannakis, Ori Lahav, and Viktor Vafeiadis. "Kater: Automating Weak Memory Model Metatheory and Consistency Checking". In: *Proceedings of the ACM on Programming Languages* 7.POPL (2023), pp. 544–572. DOI: 10.1145/3571212.

[8] Michalis Kokologiannakis and Viktor Vafeiadis. "GenMC: A Model Checker for Weak Memory Models". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Theoretical Computer Science and General Issues. Cham: Springer International Publishing and Imprint Springer, 2021, pp. 427–440. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8{\textunderscore}20. URL: https://link.springer.com/chapter/10.1007/978-3-030-81685-8_20.

[9] Aristotelis Koutsouridis, Michalis Kokologiannakis, and Viktor Vafeiadis. "Automating Memory Model Metatheory with Intersections: Schloss Dagstuhl – Leibniz-Zentrum für Informatik". In: *LIPIcs, Volume 311, CONCUR 2024* 311 (2024). DOI: 10.4230/LIPICS.CONCUR.2024.33.

[10] Ori Lahav and Udi Boker. “What’s Decidable About Causally Consistent Shared Memory?” In: *ACM Transactions on Programming Languages and Systems* 44.2 (2022), pp. 1–55. ISSN: 0164-0925. DOI: 10.1145/3505273.

[11] Saksham Sharma, Vishwanath HV, and M. Uttara Kumari. “Meow: Memory Evaluation through Optimized Workloads - Synthesizing Litmus Tests for Memory Consistency Verification”. In: *2023 IEEE 20th India Council International Conference (INDICON)*. IEEE, 2023, pp. 73–78. DOI: 10.1109/indicon59947.2023.10440892.

[12] Hünkar Can Tunç et al. “Optimal Reads-From Consistency Checking for C11-Style Memory Models”. In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023), pp. 761–785. DOI: 10.1145/3591251.

[13] John Wickerson et al. “Automatically comparing memory consistency models”. In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. Ed. by Giuseppe Castagna. ACM Digital Library. New York, NY: ACM, 2017, pp. 190–204. ISBN: 9781450346603. DOI: 10.1145/3009837.3009838.